

Integrated AI Systems for Smart Room & Air-Conditioning Control

Complete Source-Code Explanation - **src/**, launchers & integration

Course: CSE-334 | Dept. of CSE, Primeasia University | Summer 2026
Student: Md. Ragib Ashhab (242-002-042) | Supervisor: Dr. Noman Hossain
Submission: 24 September 2026 | App v2.0.0 (Modern) / v1.0.0 (Classic)

Item	Detail
Scope	Every .py file under src/ + run_gui.py + run_classic.py + setup.py
Problems covered	1) Fuzzy AC control, 2) FOPL smart-room advisor, 3) RL thermostat + data-driven extension
How to read	Each chapter = one file: purpose, classes/functions, line-by-line logic, example I/O, viva notes
Run it	python run_gui.py (modern) / python run_classic.py (classic) / python -m src --console

Pipeline used throughout this document: **Temperature / Room Conditions -> Fuzzy Logic Controller -> FOPL Smart-Room Policy -> Reinforcement Learning -> Integrated Results & Visualizations**. Temperature bands are shared: Cold <20, Comfort 20-24, Warm 24-28, Hot >=28, ExtremeHeat >=33. Fuzzy uses a coarse 3-state view (Cold/Comfortable/Hot centred at 22C); FOPL refines Warm vs Hot into AC_ECO vs AC_HIGH.

This PDF is a companion to PROJECT_REPORT.pdf. That report gives objectives and findings; this document explains **how each line of code implements those ideas** so you can defend it in viva.

Contents

1. Repository & src/ structure overview
2. Fuzzy logic - src/fuzzy_logic/fuzzy_system.py
3. Fuzzy logic - src/fuzzy_logic/rules.py
4. FOPL engine - src/fopl/knowledge_base.py
5. FOPL policy - src/fopl/advisor.py (13 rules)
6. RL environment - src/reinforcement_learning/env.py
7. RL agent (Q-learning) - src/reinforcement_learning/agent.py
8. RL trainer - src/reinforcement_learning/trainer.py
9. Data-driven - src/data_driven/generator.py & pipeline.py
10. Shared helpers - src/common/config.py, utils.py, version.py, __main__.py
11. Integration - src/main.py (run_demo + 2300-line DemoApp GUI)
12. Launchers - run_gui.py, run_classic.py, setup.py
13. End-to-end example & viva cheat-sheet

1. Repository & src/ structure overview

Root **ai-systems-project/** holds **src/** (importable package), **run_gui.py / run_classic.py / run_classic.bat / run_gui.bat** (one-click launchers), **tests/**, **docs/**, **data/**, **models/**, **requirements.txt**, **setup.py**. The four AI modules are deliberately independent so each can be unit-tested, then combined only in **src/main.py:run_demo()** and **run_classic.py:compute()**.

```
src/
__main__.py          # python -m src -> main()
version.py           # MAIN_VERSION=2.0.0, CLASSIC_VERSION=1.0.0
main.py              # run_demo() + DemoApp (modern GUI, ~2315 lines)
common/config.py, utils.py
fuzzy_logic/fuzzy_system.py, rules.py
fopl/knowledge_base.py, advisor.py
reinforcement_learning/env.py, agent.py, trainer.py
data_driven/generator.py, pipeline.py
run_gui.py           # thin wrapper -> src.main.launch_gui()
run_classic.py       # compute()+format_result()+ClassicApp (plain GUI)
tests/test_fuzzy|fopl|rl|generator|gui.py
```

Design rule: no module imports another AI module. Fuzzy never imports FOPL, RL never imports data code. Shared constants (20/24 borders) are duplicated with comments pointing at each other to keep modules decoupled but consistent.

2. Fuzzy logic - src/fuzzy_logic/fuzzy_system.py (53 lines)

Purpose: Mamdani-style temperature controller. Turns a continuous temperature into a gradual Cold/Comfortable/Hot degree, then picks the strongest set. This is Problem-1 in the brief.

2.1 Constants & state

```
COLD_MAX = 20.0
COMFORT_MAX = 24.0
COMFORT_MID = 22.0

class FuzzySystem:
    def __init__(self):
        self.temperature = 22.0
        self.current_temperature = 22.0
```

22C is the comfort centre. 20/24 are the exact borders also used in **rules.py** and **fopl/advisor.py**. Decision crossovers sit at ~19.6 and ~24.4 because of the 6-degree slopes below, i.e. effectively the 20/24 borders.

2.2 Triangular membership

```
@staticmethod
def _triangle(value, left, peak, right):
    if value <= left or value >= right: return 0.0
    if value == peak: return 1.0
    if value < peak: return (value-left)/(peak-left)
    return (right-value)/(right-peak)
```

Linear ramp up then down. Used only for **comfortable = triangle(v,18,22,26)**: 0 at 18C, 1.0 at 22C, 0 at 26C. Cold/hot use simpler linear formulas: **cold=max(0,min(1,(22-v)/6))**, **hot=max(0,min(1,(v-22)/6))**. At 16C: cold=1.0, comfortable~0.0, hot=0.0. At 22C: cold=0, comfortable=1, hot=0. At 30C: hot=1.0. Near 20C both cold and comfortable are ~0.3, so the controller degrades gracefully instead of flipping a binary switch.

2.3 Decision

```
def evaluate(self, input_value=None):
    value = self.current_temperature if input_value is None else float(input_value)
    memberships = self.membership(value)
    action_for_set = {'cold': 'Increase Temperature',
                      'comfortable': 'Maintain Temperature',
                      'hot': 'Decrease Temperature'}
    return action_for_set[max(memberships, key=memberships.get)]
```

max() = defuzzification by winner-takes-all. No centroid math needed for 3 sets. **set_temperature()** just stores float and returns e.g. *Temperature set to 22C*. Hot here covers both FOPL Warm (eco cooling) and Hot (full cooling); FOPL refines it later.

Viva line: Fuzzy is for *gradual/uncertain* temperature. A reading of 23.5C is partly comfortable, partly hot - membership captures that, a plain if/else cannot.

3. Fuzzy logic - src/fuzzy_logic/rules.py (25 lines)

Purpose: crisp, testable mirror of the fuzzy bands. Lets unit tests assert borders without floating membership math.

```
def create_fuzzy_rule(name, conditions, output):
    return {'name': name, 'conditions': conditions, 'output': output}

def create_rules():
    return [
        create_fuzzy_rule('cold', [lambda v: v < COLD_MAX], 'Increase Temperature'),
        create_fuzzy_rule('warm', [lambda v: v > COMFORT_MAX], 'Decrease Temperature'),
        create_fuzzy_rule('stable', [lambda v: COLD_MAX <= v <= COMFORT_MAX], 'Maintain
Temperature'),
    ]

def evaluate_rule(rule, inputs):
    if isinstance(inputs, dict):
        input_value = inputs.get('input_value', inputs.get('temperature', 0))
    else: input_value = inputs
    return rule['output'] if all(c(input_value) for c in rule['conditions']) else False
```

Each rule is a dict with lambdas. **evaluate_rule** accepts a raw number or dict (GUI passes dicts). Returns output string or **False** if premise fails. Example: **evaluate_rule(rules[0],16)='Increase Temperature'**, **evaluate_rule(rules[0],22)=False**. The name *warm* (not *hot*) signals it merges FOPL Warm+HOT, both need cooling.

4. FOPL engine - src/fopl/knowledge_base.py (142 lines)

Purpose: tiny general First-Order Predicate Logic kernel: terms, atoms, Horn rules with variables, forward chaining. No dependency on rooms - **advisor.py** instantiates it for rooms. This is the logic machinery for Problem-2.

4.1 Terms & atoms

```
class Variable:
    def __init__(self, name): self.name = str(name) # repr ?r
class Constant:
    def __init__(self, name): self.name = str(name) # repr R1
class Atom:
    def __init__(self, predicate, args=(), negated=False): ...
    def ground(self, binding): # {?r:R1} -> ('Hot', ('R1',))
    ...
    def describe(self): # 'Hot(?r)' or '¬EnergySaver(?r)'
```

Plain strings are treated as constants. **negated=True** means logical NOT. Grounding replaces every **Variable** present in binding, otherwise keeps its name - so partially ground atoms are still printable.

4.2 Rules & knowledge base

```
class Rule:
    def __init__(self, name, variables, premises, conclusion, description=''):
        # repr: R1-HighCooling: forall r. Hot(?r) ^ Occupied(?r) ^ ¬EnergySaver(?r) -> AC_HIGH(?r)

class KnowledgeBase:
    def __init__(self, constants=()): self.constants=[...]; self.facts=set(); self.rules=[]
    def tell(self, predicate, *args): self.facts.add((predicate, tuple(args)))
    def ask(self, predicate): return sorted(args for pred, args in self.facts if pred==predicate)
    def holds(self, ground_atom): return ground_atom in self.facts
```

4.3 Forward chaining (the inference loop)

```
def _bindings(self, variables):
    if not variables: yield {}
    else:
        for combo in product(self.constants, repeat=len(variables)):
            yield dict(zip(variables, combo))

def forward_chain(self, max_rounds=50):
    trace=[]
    for _ in range(max_rounds):
        fired=False
        for rule in self.rules:
            for binding in self._bindings(rule.variables):
                if all((not self.holds(p.ground(binding))) if p.negated
                       else self.holds(p.ground(binding)) for p in rule.premises):
                    concl=rule.conclusion.ground(binding)
                    if concl not in self.facts:
                        self.facts.add(concl); fired=True; trace.append((rule.name, binding, concl))
            if not fired: break
    return trace
```

Semantics: Horn rules, universal variables grounded over known constants (here just **R1**, so one binding per rule). Negation is **negation-as-failure under closed-world assumption**: NOT P holds when P cannot be derived. Loop runs to fixpoint (no new facts) or 50 rounds. **conclusions(predicates)** filters action atoms sorted, e.g. *AC_HIGH(R1)*. **describe_trace()** renders *R1-HighCooling {r=R1}: -> AC_HIGH(R1)* for the GUI LEDs and report export. Complexity is tiny: 13 rules x 1 constant = 13 checks per round.

5. FOPL policy - src/fopl/advisor.py (133 lines)

Purpose: real-world use case - classroom/study-room controller deciding AC, heating, lights, blinds, windows, safety alerts from sensor facts. Uses the engine above. Temperature bands intentionally match fuzzy: Cold <20, Comfort 20-24, Warm 24-28, Hot >=28, ExtremeHeat >=33 safety band.

```
ROOM='R1'
COLD_MAX=20.0; COMFORT_MAX=24.0; WARM_MAX=28.0; EXTREME_MIN=33.0
ACTION_PREDICATES=('AC_HIGH', 'AC_ECO', 'AC_OFF', 'AC_STANDBY', 'HEATER_ON',
                   'HEATER_OFF', 'LIGHTS_OFF', 'DIM_LIGHTS', 'BLINDS_DOWN', 'WINDOWS_OPEN', 'ALERT_OVERHEAT')

def temp_band(t):
    t=float(t)
    if t>=33: return 'ExtremeHeat'
    if t>=28: return 'Hot'
    if t>=24: return 'Warm'
    if t>=20: return 'Comfort'
    return 'Cold'
```

5.1 The 13 policy rules (build_rules)

ID	Premises (AND)	Conclusion	Intent
R1-HighCooling	Hot AND Occupied AND NOT Saver	AC_HIGH	Hot occupied, full comfort
R2-EcoCooling	Warm AND Occupied	AC_ECO	Warm occupied, eco cooling
R3-SaverCooling	Hot AND Occupied AND Saver	AC_ECO	Saver caps hot at eco
R4a-EmptyHotAC	Hot AND NOT Occupied	AC_OFF	Empty hot room off
R4b-EmptyHotBlinds	Hot AND NOT Occupied	BLINDS_DOWN	Block sun when empty
R5-HeatOn	Cold AND Occupied	HEATER_ON	Cold occupied heating
R6-HeatOff	Cold AND NOT Occupied	HEATER_OFF	Cold empty, heating off
R7a-ComfortStandby	Comfort AND Occupied	AC_STANDBY	Comfort occupied standby
R7b-ComfortEmpty	Comfort AND NOT Occupied	AC_OFF	Comfort empty off
R8-NightDim	Night AND Occupied	DIM_LIGHTS	Night occupied dim
R9-LightsOff	NOT Occupied	LIGHTS_OFF	Empty lights off
R10-Ventilate	Warm AND NOT Occupied AND NOT Night	WINDOWS_OPEN	Day natural vent
R11-OverheatAlert	ExtremeHeat	ALERT_OVERHEAT	Safety override

Helper `_r(name,premises,conclusion)` builds a Rule with variable `?r`; premises is a list of (predicate, positive_bool) where False becomes negated. Example from code:

`_r('R1-HighCooling',[(('Hot',True),('Occupied',True),('EnergySaver',False))], 'AC_HIGH')`.

5.2 Running it (build_advisor)

```
def build_advisor(temperature, occupied=True, night=False, energy_saver=False):
    kb=KnowledgeBase(constants=[ROOM])
    band=temp_band(temperature); kb.tell(band,ROOM)
    if band=='ExtremeHeat': kb.tell('Hot',ROOM) # extreme implies hot
    if occupied: kb.tell('Occupied',ROOM)
    if night: kb.tell('Night',ROOM)
    if energy_saver: kb.tell('EnergySaver',ROOM)
    kb.rules.extend(build_rules())
    trace=kb.forward_chain()
    return {'temperature':float(temperature),'band':band,'occupied':...,
            'facts':sorted(...),'conclusions':kb.conclusions(ACTION_PREDICATES),
            'fired':kb.describe_trace(trace)}
```

Worked examples: **30C+Occupied** -> facts Hot,Occupied -> R1 fires -> *AC_HIGH(R1)*. **30C+Occupied+Saver** -> R1 blocked by NOT Saver, R3 fires -> *AC_ECO(R1)*. **30C+Empty** -> R4a+R4b+R9 -> *AC_OFF, BLINDS_DOWN, LIGHTS_OFF*. **16C+Occupied** -> *HEATER_ON*. **22C+Night+Occupied** -> *AC_STANDBY, DIM_LIGHTS*. ExtremeHeat also asserts Hot so cooling rules still apply plus ALERT.

6. RL environment - src/reinforcement_learning/env.py (39 lines)

Purpose: tiny discrete thermostat world the agent learns in. States 0..5 (cold to comfort), goal=5. Problem-3 in the brief. Educational simulation, not a production HVAC model.

```
class RLEnvironment:
```

```

def __init__(self): self.state=0; self.done=False; self.goal=5
def reset(self): self.state=0; self.done=False; return 0
def step(self,action):
    ns=self.get_next_state(action); r=self.calculate_reward(ns)
    self.done=self.is_terminal_state(ns); self.state=ns; return (r,ns)
def get_next_state(self,action):
    if action==0: return min(self.state+1,self.goal) # cooler/warmer step
    return max(self.state-1,0)
def calculate_reward(self,state):
    if state>=self.goal: return 1.8
    d=self.goal-state
    if d<=1: return 1.0
    if d<=2: return 0.7
    return max(0.1, 0.45-(d*0.07))
def is_terminal_state(self,state): return state>=self.goal

```

Note argument order: **step returns (reward, next_state)**, not Gym's (obs,reward,done). Both trainer and GUI unpack in that order. Reward shaping encourages reaching and staying near comfort; terminal = goal reached, episode ends. Action 0 moves toward goal, 1 moves away - agent must discover that.

7. RL agent - src/reinforcement_learning/agent.py (43 lines)

Purpose: tabular Q-learning agent. Q-table maps state -> [Q(a0),Q(a1)].

```

class RLAgent:
    def __init__(self, action_space=None):
        self.action_space=[0,1] if action_space is None else list(action_space)
        self.q_table={}
    def choose_action(self, state, exploration_rate=0.0):
        key=tuple(state) if isinstance(state,list) else state
        if key not in self.q_table: self.q_table[key]=[0.0]*len(self.action_space)
        q=self.q_table[key]
        if random.random()<exploration_rate: return random.choice(self.action_space)
        if len(set(q))==1: return random.choice(self.action_space) # tie-break
        return self.action_space[q.index(max(q))] # exploit (returns action, not index)

    def learn(self, state, action, reward, next_state, learning_rate=0.2, discount_factor=0.5):
        ...
        ai=self.action_space.index(action) if action in self.action_space else 0
        best=self.q_table[nkey].index(max(self.q_table[nkey]))
        target=reward+discount_factor*self.q_table[nkey][best]
        delta=target-self.q_table[skey][ai]
        updated=self.q_table[skey][ai]+learning_rate*delta
        self.q_table[skey][ai]=max(-2.0,min(2.0,updated)) # clip for stability

```

Update is $Q(s,a) \leftarrow Q(s,a) + 0.2 * [r + 0.5 * \max_{a'} Q(s') - Q(s,a)]$. List states are tuple-ified so the same code works for scalar or vector states. **q_values** property is a backward-compat alias to **q_table**. Clipping to [-2,2] prevents divergence in the tiny demo.

8. RL trainer - src/reinforcement_learning/trainer.py (57 lines)

```
class RLTrainer:
    def __init__(self, agent, environment):
        self.agent=agent; self.environment=environment
        self.is_trained=False; self.rewards=[]
    def train(self, episodes=1, max_steps=100):
        self.rewards=[]
        for episode in range(episodes):
            s=self.environment.reset(); done=False; total=0.0; steps=0
            while not done and steps<max_steps:
                a=self.agent.choose_action(s, exploration_rate=0.1)
                reward, ns=self.environment.step(a) # note order
                done=self.environment.is_terminal_state(ns)
                self.agent.learn(s, a, reward, ns)
                s=ns; total+=float(reward); steps+=1
            self.rewards.append(round(total, 2))
        self.is_trained=True; return True
    def evaluate(self, episodes=1, max_steps=100):
        if episodes<=0: raise ValueError('episodes must be >0')
        total=0.0
        for _ in range(episodes):
            s=self.environment.reset(); done=False; steps=0
            while not done and steps<max_steps:
                a=self.agent.choose_action(s, exploration_rate=0.0) # greedy
                reward, ns=self.environment.step(a); done=...
                s=ns; total+=float(reward); steps+=1
            if not done: continue
        return total/episodes
```

train uses eps=0.1 exploration, fixed lr/discount from agent defaults. **rewards** list (one total per episode) drives the GUI reward chart with best/average/latest stats. **evaluate** is greedy, no learning, returns mean reward. GUI slider sets episodes 1-20.

9. Data-driven - src/data_driven/generator.py (48 lines) & pipeline.py (38 lines)

Purpose: extension (not counted as required problem): generate small supervised/unsupervised sets, clean, transform, save/load JSON for experimentation.

```
class DataGenerator:
    def __init__(self, method='supervised'): self.method=method
    def generate_data(self, *a, **k):
        if self.method=='supervised': return self.generate_supervised_data(*a, **k)
        if self.method=='unsupervised': return self.generate_unsupervised_data(*a, **k)
        raise ValueError("Choose 'supervised' or 'unsupervised'")
    def generate_supervised_data(self, num_samples=10):
        return [[i, i+1, 1 if i%2==0 else 0] for i in range(num_samples)]
    def generate_unsupervised_data(self, num_samples=10):
        return [[i, i*i] for i in range(num_samples)]
    def clean_data(self, raw): return [x for x in raw if x is not None]
    def transform_data(self, raw): return [self._square_item(x) for x in raw]
    @staticmethod
    def _square_item(item): # bool preserved (bool is subclass of int!)
        if isinstance(item, bool): return item
        if isinstance(item, (int, float)): return item**2
        if isinstance(item, list): return [DataGenerator._square_item(x) for x in item]
        if isinstance(item, tuple): return tuple(DataGenerator._square_item(x) for x in item)
        return item
    def save_data(self, data, filename): Path(filename).parent.mkdir(parents=True, exist_ok=True);
    ...
    def load_data(self, filename): return json.loads(Path(filename).read_text())
```

DataPipeline is the same clean+square logic but stateful: **__init__(data)**, **clean_data(data=None)**, **transform_data(data=None)**, **process(data=None)=clean->transform**, **run_pipeline()=process()**. GUI calls **DataPipeline(raw).process(raw)**. Supervised rows are [x,x+1,label]; unsupervised are [x,x^2]. Squaring

demonstrates transformation; bool check avoids turning label True into 1.

Example: supervised n=3 -> [[0,1,1],[1,2,0],[2,3,1]] -> cleaned (no-op) -> squared [[0,1,1],[1,4,0],[4,9,1]] (labels untouched).

10. Shared helpers - common/, version.py, __main__.py

10.1 src/common/config.py (43 lines)

Path + hyperparameter constants. **BASE_DIR**=Path(__file__).parents[2] (src/common/config.py -> root).

Derives **DATA_DIR**, **RAW/PROCESSED**, **MODELS_DIR** (fuzzy/rl/generator), **EXPERIMENTS_DIR**.

RL_HYPERPARAMS documents lr 0.001/discount 0.99/exploration schedule (reference values; live agent uses 0.2/0.5). **FUZZY_SETTINGS** range (16,30) default 22, **DATA_GENERATION_SETTINGS** 1000 samples.

LOGGING_LEVEL=INFO.

10.2 src/common/utils.py (41 lines)

calculate_mean/median/mode (mode returns first most-frequent for stable type), **normalize_data** min-max to 0-1 (all-equal -> zeros), **split_data(data,ratio)** train/test split with ValueError if ratio not in (0,1). Used by notebooks/tests, not the hot GUI path.

10.3 src/version.py, __main__.py, __init__.py

version.py: **MAIN_VERSION**='2.0.0' (modern GUI title), **CLASSIC_VERSION**='1.0.0', alias

__version__=**MAIN**, mirrored in setup.py. **__main__.py** (5 lines): **from .main import main; main()** so **python**

-m src [--console|--gui] works. Package **__init__.py** files are one-line markers, except common which is empty.

11. Integration - src/main.py (2315 lines)

Purpose: headless demo + full modern GUI. Imports with dual try/except (flat **fuzzy_logic.*** vs **src.fuzzy_logic.***) so it runs as script or package, frozen exe or dev. Optional tkinter/matplotlib guarded by **TK_AVAILABLE / FIGURE_AVAILABLE**.

11.1 Headless functions

```
def normalize_values(items): # flatten numbers/lists/tuples/dicts -> [floats] or [0.0]
def run_demo(temp_value=22, rl_episodes=5, data_points=5, data_method='supervised'):
    fuzzy=FuzzySystem(); fuzzy.set_temperature(temp_value); fr=fuzzy.evaluate(temp_value)
    env=RLEnvironment(); agent=RLAgent([0,1]); trainer=RLTrainer(agent,env)
    trainer.train(episodes=rl_episodes,max_steps=100)
    gen=DataGenerator(method=data_method); raw=gen.generate_data(num_samples=data_points)
    processed=DataPipeline(raw).process(raw)
    return {'temperature':..., 'fuzzy_result':fr, 'processed_items':len(processed), ...,
            'rl_rewards':trainer.rewards, 'processed_data':processed}
def console_demo(): print(run_demo() summary)
def launch_gui(): DemoApp().mainloop() with friendly missing-dep messages
def main(): argparse --gui/--console/--no-gui -> console_demo() or launch_gui()
```

Note: **run_demo** omits FOPL (GUI calls **build_advisor** separately for live switches); **run_classic.compute** includes it.

11.2 DemoApp GUI (the bulk of main.py)

Class **DemoApp(tk.Tk):247** with theme dicts dark/light, helpers **_rounded_rect**, **RoundedButton(Canvas)**, **RoundedPanel(Frame)** for modern rounded cards (outline="" avoids black arc ticks; inner packed not placed or panels collapse to 1px - see code comments). Layout: header (title + power + theme + status dot) -> sky weather strip -> Controls card (3 sliders + method combo + Run/Reset/Export/Auto + Cold/Normal/Hot presets 16/8/10, 22/5/5, 30/12/15) -> FOPL POLICY LED row (11 LEDs: HIGH/ECO/A-OFF/STBY/HEAT/H-OFF/L-OFF/DIM/BLIND/WIN/ALERT + Occupied/Night/Saver checkbuttons + caption) -> Results grid (Fuzzy / RL reward / Data charts via matplotlib FigureCanvasTkAgg + hover readouts + rl_stats/data_stats) -> footer.

Behaviour: sliders trigger **_on_temperature_changed / _on_settings_changed / _refresh_fopl / _refresh_all**; keyboard Up/Down temp, Left/Right episodes, R/Space run, Esc reset; smooth mouse-wheel glide; animated border sweep (**_paint_frame**), pulsing status dot, cycling weather (snow/rain/sun/leaves/petals drawn on canvas from temp + phase), splash cover, fade in/out on theme toggle (rebuilds UI). **_resource_path/ apply_window_icon** handle PyInstaller _MEIPASS assets. Power OFF disables widgets and shows STANDBY. Export writes report text.

12. Launchers - run_gui.py (20 lines), run_classic.py (499 lines), setup.py

12.1 run_gui.py

```
ROOT=Path(__file__).parent; SRC=ROOT/'src'
sys.path.insert(0, str(SRC)); sys.path.insert(0, str(ROOT))
from src.main import console_demo, launch_gui
if __name__=='__main__':
    --console/--no-gui -> console_demo(); --help -> usage; else launch_gui()
```

Double-click opens GUI every time; no other logic.

12.2 run_classic.py

Plain white fallback GUI + headless compute.

compute(temp,episodes,points,method,occupied,night,saver):80 runs all four modules including **build_advisor**, flattens processed data (skipping bools), returns dict with fuzzy_result, rewards/best/average, flat/min/max/mean/count, band, conclusions. **format_result():132** renders 8-line text summary.

ClassicApp(tk.Tk):152 - header + 3 Scales + OptionMenu + 3 checkbuttons + Run/Reset + Text output + 3 matplotlib charts (bar/scatter/line with mean/best overlays) + 11 LEDs (oval fill white vs colour).

Keyboard/power/reset mirror modern app. **main():483** falls back to printing **compute()** if tkinter missing or window fails.

12.3 setup.py (32 lines)

setuptools: name **ai-systems-project v2.0.0**, author Md. Ragib Ashhab, **find_packages(where='src')**, **package_dir={'': 'src'}**, install_requires from requirements.txt, python>=3.11, MIT classifiers. Version mirrors **version.MAIN_VERSION**.

13. End-to-end example & viva cheat-sheet

Input: **30C, Occupied, Day, No Saver, 12 RL episodes, 15 supervised points**. Fuzzy: membership hot~1.0 -> *Decrease Temperature*. FOPL: facts Hot(R1),Occupied(R1) -> trace R1-HighCooling -> *AC_HIGH(R1)*. RL: agent starts 0, action 0 repeatedly, rewards ~[0.1..1.8], chart rises, best~last. Data: 15 rows generated, cleaned, squared, plotted with mean line. GUI shows all four simultaneously; LEDs highlight HIGH.

If asked	Say (one line each)
Why fuzzy?	Continuous temp needs gradual sets; 23.5C is partly comfortable.
Why FOPL?	Explicit rules + safety + explanation; trace shows which rule fired.
Why RL?	Learns control from reward interaction, no hand thresholds.
Bands consistent?	Yes: 20/24 shared; fuzzy Hot = FOPL Warm+Hot; FOPL refines to ECO vs HIGH.
Negation?	Negation-as-failure, closed-world: NOT P = P not derivable.
Q update?	$Q += 0.2 * (r + 0.5 * \max Q' - Q)$, clipped $[-2, 2]$, eps 0.1 train / 0 greedy eval.
Limitations?	Simulated sensors, simplified RL, no humidity/pricing, no hardware control.

Appendix A - File mapping (from PROJECT_REPORT, verified)

Area	Files	Test
Fuzzy	src/fuzzy_logic/fuzzy_system.py src/fuzzy_logic/rules.py	tests/test_fuzzy.py
FOPL	src/fopl/knowledge_base.py src/fopl/advisor.py	tests/test_fopl.py
RL	src/reinforcement_learning/env.py src/reinforcement_learning/agent.py src/reinforcement_learning/trainer.py	tests/test_rl.py
Data	src/data_driven/generator.py src/data_driven/pipeline.py	tests/test_generator.py
GUI+integration	src/main.py run_gui.py run_classic.py	tests/test_gui.py + confestest.py
Shared	src/common/config.py src/common/utils.py src/version.py src/__main__.py	-

Generated from live src/ on this machine. For full code listings see the repository; this PDF explains logic file-by-file for report viva. Originality: room rules, software structure, interface and integration are project-specific; fuzzy/FOPL/RL concepts are standard academic methods.